

Agrofarm

**Matheus Felipe¹, Fernanda Soares², Gabriel Lacerda³,
Júlia Rocha⁴, Alice Shikida⁵, Matheus Dias⁶**

¹Instituto de Ciências Exatas e Informática
Pontifícia Universidade Católica de Minas Gerais (PUC Minas)
Belo Horizonte – MG – Brasil

{matheus.silva.882564, fernanda.cunha.1366648, gabriel.lacerda}@sga.pucminas.br

{jfiorini, alice.salomao, matheus.mendes}@sga.pucminas.br

Resumo. *Escrever aqui o resumo. O resumo deve contextualizar rapidamente o trabalho, descrever seu objetivo e, ao final, mostrar algum resultado relevante do trabalho (até 10 linhas).*

1. Introdução

O agronegócio consolidou-se como um pilar essencial para a economia global, impulsionado por um processo contínuo de modernização. Recentemente, a ascensão de tecnologias digitais e sistemas de gestão agrícola transformou o setor, permitindo a organização minuciosa de informações, a otimização de processos produtivos e o suporte estratégico à tomada de decisão. Essas ferramentas são fundamentais para estruturar as atividades no campo, reduzindo falhas operacionais e elevando a eficiência da produção.

Entretanto, apesar dessa evolução tecnológica, ainda persiste uma parcela considerável de produtores que operam de forma totalmente informal. No caso específico do cliente deste projeto, a gestão de atividades críticas, como, plantio, colheita, controle de insumos e vendas, é realizada sem qualquer registro formal. Atualmente, o fluxo de dados depende exclusivamente da memória e da experiência acumulada, o que caracteriza uma administração baseada majoritariamente na intuição.

Essa ausência de registros estruturados acarreta riscos significativos à sustentabilidade do negócio. A dependência da memória dificulta o controle financeiro rigoroso e o acompanhamento preciso da produtividade, tornando árdua a tarefa de identificar gargalos ou oportunidades de melhoria. Além disso, a volatilidade das informações não registradas compromete o planejamento a longo prazo e expõe o produtor a perdas de dados vitais para a continuidade operacional.

Diante desse cenário, o cliente, um agricultor com vasto conhecimento prático, mas com dificuldades na organização digital busca uma transição para um modelo mais profissional. A solução proposta deve ser, acima de tudo, simples e acessível, permitindo o registro cotidiano da produção sem exigir conhecimentos técnicos avançados. O objetivo central é oferecer uma ferramenta intuitiva que transforme dados esparsos em uma visão clara e estratégica do negócio.

2. Objetivo

Desenvolver um sistema de gestão agrícola simples e eficiente que permita ao agricultor registrar, organizar e acompanhar suas atividades, substituindo o controle baseado apenas na memória.

2.1. Objetivos específicos

- Permitir o registro das atividades agrícolas (plantio, colheita, uso de insumos);
- Organizar informações que antes estavam apenas na memória do produtor;
- Auxiliar no controle de custos e receitas;
- Facilitar o acompanhamento da produção ao longo do tempo;
- Oferecer uma interface simples e de fácil uso;
- Apoiar a tomada de decisão com base em dados registrados.

3. Justificativa

A gestão baseada exclusivamente na memória representa um grande risco para a organização e o crescimento da produção agrícola. A ausência de registros dificulta o controle financeiro, o acompanhamento da produtividade e a identificação de melhorias, tornando o processo dependente apenas da experiência do produtor e mais suscetível a erros.

Diante disso, o desenvolvimento de um sistema de gestão agrícola se torna essencial para transformar informações informais em dados estruturados, permitindo maior controle, planejamento e tomada de decisão. A proposta deste projeto também está alinhada aos Objetivos de Desenvolvimento Sustentável (ODS), especialmente:

- ODS 2 – Fome Zero e Agricultura Sustentável: ao contribuir para uma produção mais organizada e eficiente, promovendo melhor aproveitamento dos recursos e aumento da produtividade;
- ODS 8 – Trabalho Decente e Crescimento Econômico: ao auxiliar na gestão financeira e na viabilidade do negócio rural, contribuindo para a geração de renda e sustentabilidade econômica do produtor;
- ODS 12 – Consumo e Produção Responsáveis: ao incentivar a otimização do uso de insumos e a redução de desperdícios por meio do controle e análise das informações.

Além do impacto no contexto agrícola, o desenvolvimento deste projeto também é fundamental para a formação acadêmica dos alunos envolvidos. A construção da solução permite aplicar, na prática, conhecimentos de Engenharia de Software, como levantamento de requisitos, modelagem de sistemas, desenvolvimento e validação de soluções.

Também promove o desenvolvimento de habilidades importantes, como pensamento crítico, resolução de problemas reais, trabalho em equipe e contato com um cliente real, aproximando os alunos do mercado de trabalho e de desafios concretos da área de tecnologia.

Dessa forma, o projeto não apenas resolve uma necessidade real do cliente, mas também contribui diretamente para a formação profissional dos estudantes, unindo teoria e prática em um contexto aplicado.

4. Metodologia

Para a concepção e construção do Agrofarm, adotou-se uma abordagem baseada em desenvolvimento ágil de software e na separação clara de responsabilidades arquiteturais. O sistema foi estruturado em uma arquitetura cliente-servidor (Frontend e Backend), garantindo escalabilidade, manutenibilidade e segurança.

4.1. Arquitetura e Tecnologias

A aplicação foi dividida em duas frentes independentes que se comunicam via requisições HTTP (API RESTful).

O **Backend** foi desenvolvido em Node.js com o framework Express, seguindo uma variação aprimorada do padrão de projeto MVC (Model-View-Controller). Para garantir o isolamento das regras de negócio e a facilidade de testes, foram adicionadas as camadas de **Service** (responsável pelas validações e lógica central) e **Repository** (responsável exclusiva pelas operações de banco de dados). O acesso aos dados é gerenciado pelo **ORM Prisma** (versão 5), conectado a um banco de dados relacional **PostgreSQL** hospedado na plataforma **Neon**. A segurança e validação são garantidas pelo uso de **JWT** para autenticação, **Bcrypt** para criptografia de senhas e **Zod** para a validação rigorosa dos esquemas de dados de entrada. Adicionalmente, o backend integra-se à **Evolution API** para disparos automatizados de mensagens via **WhatsApp** (usando **node-cron** para agendamentos) e à **API do Google Gemini** para processamento de inteligência artificial.

O **Frontend** foi construído como uma **Single Page Application** (SPA) utilizando **React** (versão 19) e o bundler **Vite**. A interface do usuário adota uma arquitetura baseada em componentes, estilizada através do framework utilitário **Tailwind CSS**. O fluxo de dados no lado do cliente é otimizado pelo uso do **TanStack Query** para o gerenciamento de estados assíncronos (requisições, **cache** e sincronização com o servidor) e do **Zustand** para o estado global da aplicação (como armazenamento de sessão do usuário e fazenda selecionada). As comunicações com a **API** são realizadas pela biblioteca **Axios**, e a renderização de dados estatísticos é apoiada pela biblioteca **Recharts**. O ecossistema de ambas as frentes (front e back) conta ainda com a ferramenta **Vitest** para a garantia de qualidade por meio de testes unitários e de integração.

4.2. Processo de Desenvolvimento

O gerenciamento do projeto foi norteado pelas metodologias ágeis, com adoção do framework **Scrum**, que preconiza o desenvolvimento iterativo, incremental e a adaptação contínua [Schwaber and Sutherland 2020]. O ciclo de vida do desenvolvimento foi dividido em **Sprints** quinzenais, permitindo entregas funcionais de valor e avaliações constantes dos requisitos. O escopo do projeto foi distribuído ao longo das seguintes Sprints:

Tabela 1. Entregas das Sprints

Nº Sprint	Nome	Objetivo
Sprint 1	Lapidação dos Requisitos e Design	Foco na concepção do projeto e entendimento do problema. Realização do levantamento e elaboração de documentos base, incluindo atas de reunião. Modelagem do sistema através da criação do diagrama de casos de uso e do diagrama entidade-relacionamento (DER), além da elaboração dos protótipos de tela para a validação inicial da interface.
Sprint 2		
Sprint 3		
Sprint 4		
Sprint 5		

5. Resultados

Resultados do trabalho devem ser apresentados. Consiste da descrição técnica da solução desenvolvida, incluindo a lista de requisitos (pode ser uma tabela). Use figuras e tabelas sempre que necessário.

Apresente também as principais telas da aplicação e uma explicação de como usá-las. O código fonte deve ser disponibilizado em um repositório público no Github-Classroom. O link para o repositório deve estar no Trabalho. Colocar também o link da aplicação.

Veja os exemplos de uso de Figuras e Tabelas. Todas as figuras e tabelas devem ser referenciadas no texto. Por exemplo, deve haver uma frase assim “A Figura 1 mostra ...” ou “A Tabela 2 mostra...”



Figura 1. A typical figure

Tabela 2. Variables to be considered on the evaluation of interaction techniques

	Chessboard top view	Chessboard perspective view
Selection with side movements	6.02 ± 5.22	7.01±6.84
Selection with in- depth movements	6.29±4.99	12.22±11.33
Manipulation with side movements	4.66± 4.94	3.47±2.20
Manipulation with in- depth movements	5.71 ±4.55	5.37 ±3.28

Link do vídeo:

Link do repositório:

Link da apresentação:

6. Conclusões e trabalhos futuros

A conclusão deve iniciar resgatando o objetivo do trabalho e os principais resultados alcançados. Em seguida, devem ser apresentados os trabalhos futuros.

Referências

Schwaber, K. and Sutherland, J. (2020). *The Scrum Guide*. Scrum.org.